

# CSCI 112

## Programming with C

### LECTURE: 15

J. L. Pach

# OUTLINE:

- Introduction to File Access:
  - Text
  - Binary

# INTRODUCTION TO FILE ACCESS

- **Why File Access Matters**:
  - Storing and retrieving data beyond program execution.
- Types of File Modes:
  - **Text Mode**: Reads and writes data as readable characters, char-by-char, line-by-line, etc.
  - **Binary Mode**: Reads and writes data as raw bytes, allowing efficient handling of complex data structures.

# OPENING AND CLOSING FILES 1/2

- **Function**: `fopen()`
  - Syntax: `FILE *fopen(const char *filename, const char *mode);`
  - Error Handling: Always check if `fopen()` returned `NULL`, indicating failure (e.g., file not found).
- **Function**: `fclose()`
  - Syntax: `int fclose(*FILE);`
  - Error Handling:
    Always check if `fclose()` returned `NULL`, indicating failure (e.g., file not found).

# OPENING AND CLOSING FILES 2/2

- In the context of file access, the `fopen()` and `fclose()` functions serve a role analogous to curly braces `{`, `}` in defining a code block. Opening a file with `fopen()` is akin to entering a new scope of operations on that file, similar to how an opening curly brace signals the beginning of a new block of statements. Conversely, `fclose()` marks the end of this scope, closing the file and thus concluding the block, much like a closing curly brace.
- While the compiler ensures that code blocks are properly closed, it is the programmer's responsibility to ensure files are closed correctly. An unclosed file is like an unclosed curly brace - it can lead to unexpected errors and hinder further operations.

# OPENING FILES

**Function**: `fopen()`:

```
FILE *fopen(const char *filename, const char *mode);
```

```
          /* ┌──────────────────────────────────────┬──────────────────────────────────────┐ */
          /* │    file path/file name               │       mode                           │ */
          /* ├──────────────────────────────────────┼──────────────────────────────────────┤ */
FILE *fopen(  /* │ */ const char *filename,/* │ */ const char *mode  /* │ */ );
          /* └──────────────────────────────────────┴──────────────────────────────────────┘ */
```



**Error Handling**:

Always check if `fopen()` returned `NULL`, indicating failure (e.g., file not found).

## `const char *filename` 1/3

- The `const char *filename` argument in the `fopen()` function can specify three types of file locations:

1. **File Name Only**:
    i. When only the file name is provided, `fopen()` attempts to open the file in the same directory as the currently executing program.
    ii. Example: `fopen("data.txt", "rt")` will try to open the file `data.txt` in the directory where the program is located.
2. **Relative Path**
3. **Absolute Path**

## `const char *filename` 2/3

- The `const char *filename` argument in the `fopen()` function can specify three types of file locations:

1. **File Name Only**
2. **Relative Path**:
    i. A relative path specifies the location of the file relative to the current working directory.
    ii. Example: `fopen("data/results.txt", "wt")`* will try to open the file `results.txt` in a subdirectory named `data` within the current working directory.
3. **Absolute Path**

> *Platform-specific path separators: The specific character used to separate directories in a path (e.g., / in Unix-like systems, \ \ in Windows) depends on the operating system.

## `const char *filename` 3/3

- The `const char *filename` argument in the `fopen()` function can specify three types of file locations:

1. **File Name Only**
2. **Relative Path**
3. **Absolute Path**:
    i. An absolute path provides the complete path to the file, starting from the root directory of the file system.
    ii. Example: `fopen("/home/user/documents/project/data.txt", "at")` will open the file `data.txt` in the specified directory, regardless of the current working directory.

# File Access Modes in C

## Access Modes:

| Mode | Description |
|---|---|
| r | Opens an existing file for reading only. |
| w | Opens a file for writing. Creates a new file or clears the content of an existing one. |
| a | Opens a file for appending data. Creates the file if it does not already exist. |
| x | Creates the file if it does not already exist. Fails if the file already exists. |
| + | Combined with r, w, a, or x, allows both reading and writing. |

## File Types:

| Type Modifier | Description |
|---|---|
| t | Text mode (default). Treats file as a sequence of characters. |
| b | Binary mode. Treats file as a sequence of bytes with no translation. |

# BINARY FILE I/O FUNCTIONS

- For reading blocks of binary data.
  Syntax: `int fread(void *ptr, int size, int count, FILE *stream);`
- For writing blocks of binary data.
  Syntax: `int fwrite(const void *ptr, int size, int count, FILE *stream);`
- `ptr`: Pointer to the data you want to write(read).
- `size`: Size, in bytes, of each element to be written(read).
- `count`: Number of elements to write(read), which is the third argument.
- `stream`: File pointer to the open file where data should be written(read).

# Example – Binary file

```
FILE *file;
file =  fopen("binaryFile.bin", "wb");
if(file)
{
  int x = 5;
  _ =  fwrite(&x, sizeof(x), 1, file);

  char text[10] = "Some text";
  _ =  fwrite(text, sizeof(*text), 10, file);

  float fnumber = 3.14f;
  _ =  fwrite(&fnumber, sizeof(fnumber), 1, file);
  fclose(file);
}
else
    printf("Error");
```

```
binaryFile.bin:
   Some text ĂőH@
```

```
FILE *file;
file =  fopen("binaryFile.bin", "rb");
if(file)
{
    int x;
    _ =  fread(&x, sizeof(x), 1, file);

    char text[10];
    _ =  fread(text, sizeof(*text), 10, file);

    float fnumber;
    _ =  fread(&fnumber, sizeof(fnumber), 1, file);
    fclose(file);
    printf("%d %s %f", x, text, fnumber);
}
else
    printf("Error");
```

```
Result:
15 Some text 3.140000
```

# `fseek()` function

- The function `fseek()` moves the file pointer to a specified location, allowing you to read from or write to a specific part of the file.
  Syntax: `int fseek(FILE *stream, long offset, int origin);`
- `stream`: A pointer to the `FILE` object that identifies the file.
- `offset`: The number of bytes to move the file pointer.
- `origin`: The starting position for the offset; it can be one of the following constants:
  - `SEEK_SET`: Start from the beginning of the file (`0`).
  - `SEEK_CUR`: Start from the current position of the file pointer (`1`).
  - `SEEK_END`: Start from the end of the file(`2`).

## `ftell()` and `rewind()` functions

- The function `ftell()` returns the current file position as a long integer. If an error occurs, it returns `-1`.

  Syntax: `long ftell(FILE *stream);`

- Unlike `fseek()` and `ftell()`, `rewind()` is simpler to use and is intended to reset the file pointer to the beginning of the file.

  **Syntax:** `void rewind(FILE *stream);`

# 1 example – fseek()

```c
#include <stdio.h>
int main(int argc, char *argv[])
{
   FILE *file;
   file =  fopen("binaryFile.bin", "rb");
   if(file)
   {
      /* Move to the 5th byte from the beginning */
      fseek(file, 5, SEEK_SET);
      printf("Position after fseek: %ld\n", ftell(file));

   /* Move forward by 10 bytes from the current position */
      fseek(file, 10, SEEK_CUR);
      printf("Position after another fseek: %ld\n", ftell(file));

   /* Move to the end of the file */
      fseek(file, 0, SEEK_END);
      printf("Position at the end of file: %ld\n", ftell(file));
      fclose(file);
   }
   else
      printf("Error");
   return 0;
}
```

Result:

Position after fseek: 5

Position after another fseek: 15

Position at the end of file: 18

```c
FILE *file;
file =  fopen("binaryFile.bin", "rb");
if(file)
{
   int x; int position;
   position = ftell(file);
   printf("The position in file = %d\n", position);
   _ =  fread(&x, sizeof(x), 1, file);
   position = ftell(file);
   printf("The position in file = %d\n", position);

   char text[10];
   _ =  fread(text, sizeof(*text), 10, file);
   position = ftell(file);
   printf("The position in file = %d\n", position);

   float fnumber;
   _ =  fread(&fnumber, sizeof(fnumber), 1, file);
   position = ftell(file);
   printf("The position in file = %d\n", position);
   printf("%d %s %f\n", x, text, fnumber);

   /* Move to the 0th byte from the beginning */
   fseek(file, 0, SEEK_SET);
   printf("Position after fseek(): %ld\n", ftell(file));

   x = 0;
   _ =  fread(&x, sizeof(x), 1, file);
   printf("%d\n", x);
   fclose(file);
}
else
   printf("Error");
```

Result:

```
The position in file = 0
The position in file = 4
The position in file = 14
The position in file = 18
5 Some text 3.140000
Position after fseek(): 0
5
```

# TEXT FILE I/O FUNCTIONS

- **For reading text from file**:

  `fgetc()`: Reads a single character from a file.

  `fgets()`: Reads a line from a file.

  `fscanf()`: Reads formatted input from a file (similar to `scanf()`).

- **For writing text for file**:

  `fputc()`: Writes a single character to a file.

  `fputs()`: Writes a string to a file.

  `fprintf()`: Prints formatted output to a file (similar to `printf()`).

# Reading Text from a file

1. `int fgetc(FILE *stream)`
   i. **Description**: Reads a single character from the specified file stream.
   ii. **Return Type**: Returns the character read as an unsigned char cast to an int, or `EOF` on end of file or error.
   iii. **Example**: `int ch = fgetc(file_pointer);`
2. `char *fgets(char *str, int n, FILE *stream)`
   i. **Description**: Reads a line from the file and stores it in the character array str.
   ii. **Return Type**: Returns str on success, or `NULL` on error or when end of file occurs.
   iii. **Example**: `char *result = fgets(buffer, sizeof(buffer), file_pointer);`
3. `int fscanf(FILE *stream, const char *format, ...)`
   i. **Description**: Reads formatted input from a file based on the format string, similar to `scanf`.
   ii. **Return Type**: Returns the number of items successfully read, or `EOF` if an error or end of file occurs before any items are matched.

# Writing Text to a file

1. `int fputc(int char, FILE *stream)`
   i. **Description**: Writes a single character to the specified file stream.
   ii. **Return Type**: Returns the written character as an unsigned char cast to an int, or EOF on error.
   iii. **Example**: `int result = fputc('A', file_pointer);`
2. `int fputs(const char *str, FILE *stream)`
   i. **Description**: Writes a string to the file.
   ii. **Return Type**: Returns a non-negative number on success, or EOF on error.
   iii. **Example**: `int result = fputs("Hello, World!\n", file_pointer);`
3. `int fprintf(FILE *stream, const char *format, ...)`
   i. **Description**: Writes formatted output to the specified file stream, similar to printf().
   ii. **Return Type**: Returns the number of characters written, or a negative value if an error occurs.
   iii. **Example**: `int chars_written = fprintf(file_pointer, "Integer: %d, Float: %f\n", int_var, float_var);`

# THANK

**You!**